

```
In [3]: %matplotlib ipynb
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from matplotlib.backends.backend_pdf import PdfPages

np.set_printoptions(legacy='1.25') #removes the funny printing
```

Pandas

Pictures are for fun. No need to add them to your notebook!

```
In [3]: from IPython.display import Image
Image(filename='PeterPanda2-phineas-pherb.jpg',width=400)
```

Out[3]:



Automated Pandas?

```
In [4]: from IPython.display import Image
Image(filename='Pandaborg.jpg',width=400)
```

Out[4]:



Today we want to automate a few things in pandas. this means we will need to clean up our coding and call things in a better method. I will walk you through it. Lets read in our file and get started. We will use well_data.csv again. Look on courseworks and you will see our goal is making a file with the graphs we want. I kept the df in the name as it is a dataframe

```
In [5]: df=pd.read_csv('well_data.csv')
```

Lets print the arsenic data again

```
In [6]: df['As'].head()
```

```
Out[6]: 0      NaN
        1      NaN
        2      NaN
        3    78.97747
        4      NaN
        Name: As, dtype: float64
```

Now instead of using 'As' lets set i to 'As' and try and call it. **Remember** I am using head() to save space printing. You do not need it for class.

```
In [8]: i='As'
        df[i].head()
```

```
Out[8]: 0      NaN
        1      NaN
        2      NaN
        3    78.97747
        4      NaN
        Name: As, dtype: float64
```

We can also do fucntions on it. Lets call dtype which tells us the data type.

```
In [4]: i='As'
        df[i].dtype
```

```
Out[4]: dtype('float64')
```

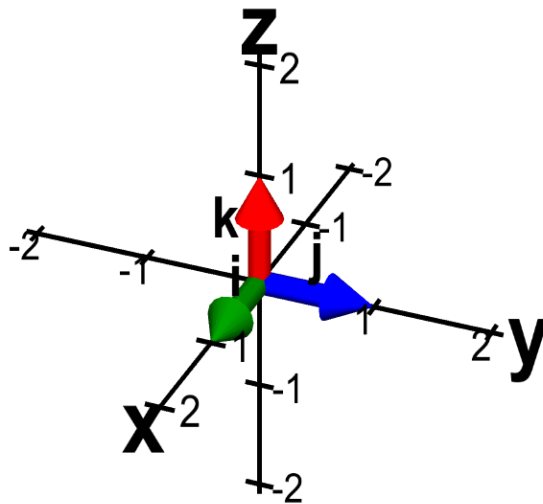
now try setting i to Drink

```
In [9]:
```

```
Out[9]: dtype('O')
```

```
In [8]: from IPython.display import Image
        Image(filename='standard_unit_vectors_3d.png',width=400)
        #copied from http://mathinsight.org/cross\_product\_formula
```

Out[8]:



I am not going to go into it. But i, j, k are used for vectors in math and are thus the common names people iterate over. So i, j, k is the vector in x, y, z space. But you could call it anything. But i, j, k are simple and easy

```
In [10]: I_need_to_think_of_a_better_name='As'
df[I_need_to_think_of_a_better_name]
```

```
Out[10]: 0      NaN
1      NaN
2      NaN
3    78.97747
4      NaN
...
754    0.13000
755   17.39000
756   112.37000
757   248.93000
758      NaN
Name: As, Length: 759, dtype: float64
```

now what if we wanted to print the datatypes one by one. we could use a for loop. The nice thing about Pandas is you can iterate over the columns automatically.

```
In [11]: for i in df:
          print (i)
```

Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
S04
Br

I used to call it col for i but this year I am going to try column

```
In [72]: for column in df:  
         print (column)
```

Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
S04
Br

Now that you know how to iterate can you go through and print each data type?

```
In [41]:
```

```

Well_ID is of type int64
Lat is of type float64
Lon is of type float64
Depth is of type int64
Drink is of type object
Si is of type float64
P is of type float64
S is of type float64
Ca is of type float64
Fe is of type float64
Ba is of type float64
Na is of type float64
Mg is of type float64
K is of type float64
Mn is of type float64
As is of type float64
Sr is of type float64
F is of type float64
Cl is of type float64
S04 is of type float64
Br is of type float64

```

We could use our format to make it cleaner and look prettier.

In [43]:

```

Well_ID is of type int64
Lat is of type float64
Lon is of type float64
Depth is of type int64
Drink is of type object
Si is of type float64
P is of type float64
S is of type float64
Ca is of type float64
Fe is of type float64
Ba is of type float64
Na is of type float64
Mg is of type float64
K is of type float64
Mn is of type float64
As is of type float64
Sr is of type float64
F is of type float64
Cl is of type float64
S04 is of type float64
Br is of type float64

```

Now lets get our code for printing a graph and instead of saying As. Lets put in i. this is my original HW answer.

THERE IS A CONFUSING PART HERE!

1. We are using arsenic twice in the same graph
2. We are plotting it
3. We are also coloring the dots with it.
4. I would keep the coloring of the dots hardwired with arsenic.
5. I use different colors I like a little more
6. Alpha is how see through the points are. 1=solid 0=invisible. Sometimes an alpha of 0.5 or 0.75 smoothes the points a bit

In [25]:

```

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

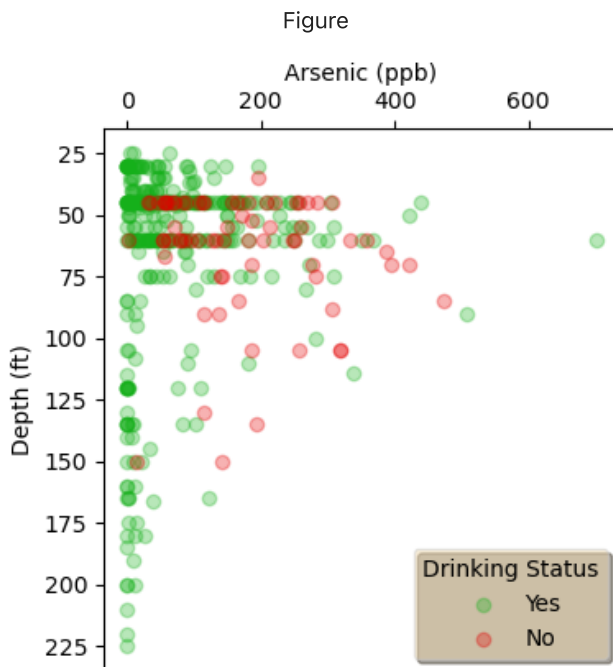
#plot may parameters
ax.scatter(df[df.Drink=='Y'].As,df[df.Drink=='Y'].Depth
          ,alpha=0.3,label='Yes',color='xkcd:green')

```

```
ax.scatter(df[df.Drink=='N'].As,df[df.Drink=='N'].Depth,
           ,alpha=0.3,label='No',color='xkcd:red')

#make look nice
ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel('Arsenic (ppb)')
ax.set_ylabel('Depth (ft)')
ax.legend(loc='best',title='Drinking Status'
          ,fancybox=True,framealpha=0.5
          ,facecolor='wheat',shadow=True
          ,draggable=True)
```

Out[25]: <matplotlib.legend.Legend at 0x145bd8f90>



If you had to drill a well and you were worried about arsenic being over 10 and making the water unsafe what depth would you drill to? What depth has no high arsenic samples?

To save just one figure you can right click on an image. But even easier is adding a line of code. I always add this line

```
fig.savefig('Name_of_figure.jpg',dpi=200,bbox_inches='tight')
```

- name the figure and the ending sets the type. jpg,gif,pdf.....
- dpi is the dots per inch and is the resolution. For a fancy figure increase to 600 or so.
- bbox_inches='tight' is this weird key word you need. If you don't have this and you have a legend, the legend gets cut off.
- It just puts the file in your working directory. Try it.

```
In [22]: fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

#plot may parameters
ax.scatter(df[df.Drink=='Y'].As,df[df.Drink=='Y'].Depth,
           ,alpha=0.5,label='Yes',color='xkcd:green')
ax.scatter(df[df.Drink=='N'].As,df[df.Drink=='N'].Depth,
           ,alpha=0.5,label='No',color='xkcd:red')

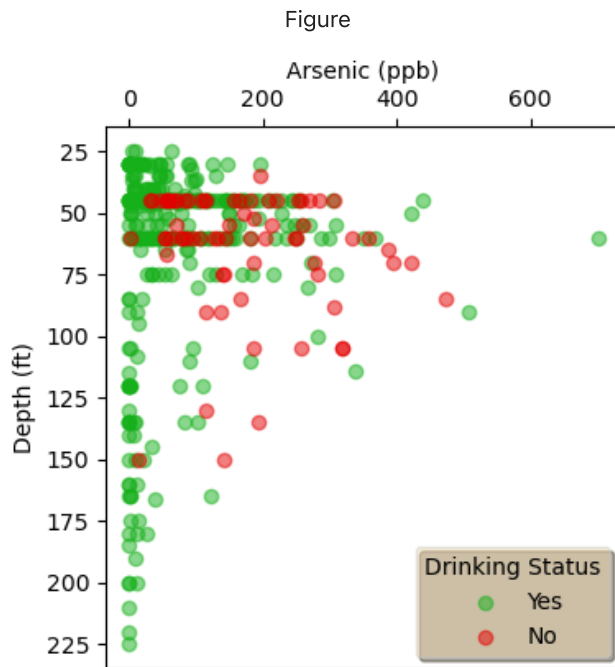
#make look nice
```

```

ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel('Arsenic (ppb)')
ax.set_ylabel('Depth (ft)')
ax.legend(loc='best', title='Drinking Status'
          , fancybox=True, framealpha=0.5
          , facecolor='wheat', shadow=True)

fig.savefig('Name_of_figure.jpg', dpi=200, bbox_inches='tight')

```



In []:

Now change it to plot iron Fe but keep the drink no drink colors

```

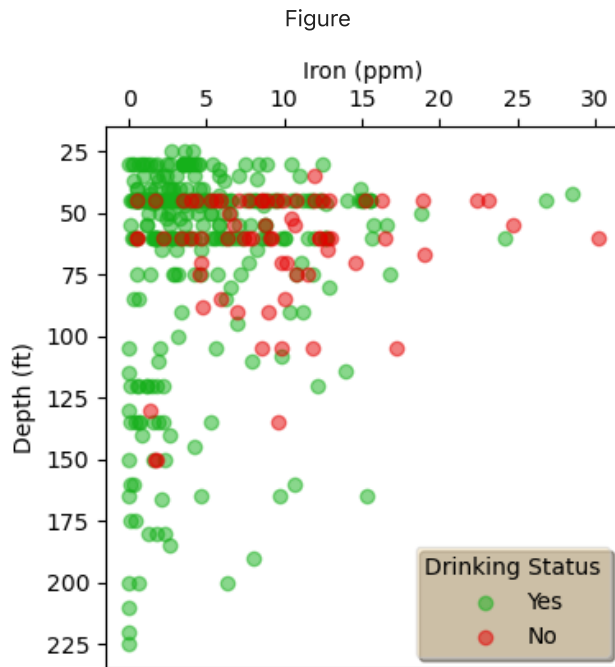
In [23]: fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

#plot may parameters
ax.scatter(df[df.Drink=='Y'].Fe,df[df.Drink=='Y'].Depth
          ,alpha=0.5,label='Yes',color='xkcd:green')
ax.scatter(df[df.Drink=='N'].Fe,df[df.Drink=='N'].Depth
          ,alpha=0.5,label='No',color='xkcd:red')

#make look nice
ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel('Iron (ppm)')
ax.set_ylabel('Depth (ft)')
ax.legend(loc='best', title='Drinking Status'
          , fancybox=True, framealpha=0.5
          , facecolor='wheat', shadow=True)

fig.savefig('Name_of_figure.jpg', dpi=200, bbox_inches='tight')

```



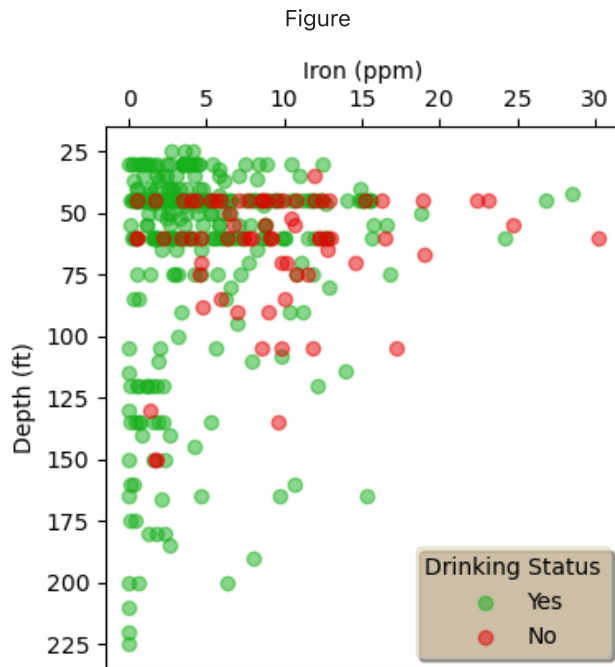
Now change .Fe to ['Fe']

```
In [26]: fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

#plot may parameters
ax.scatter(df[df.Drink=='Y']['Fe'],df[df.Drink=='Y'].Depth,
           ,alpha=0.5,label='Yes',color='xkcd:green')
ax.scatter(df[df.Drink=='N']['Fe'],df[df.Drink=='N'].Depth,
           ,alpha=0.5,label='No',color='xkcd:red')

#make look nice
ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel('Iron (ppm)')
ax.set_ylabel('Depth (ft)')
ax.legend(loc='best',title='Drinking Status'
         , fancybox=True, framealpha=0.5
         ,facecolor='wheat',shadow=True)

fig.savefig('Name_of_figure.jpg',dpi=200,bbox_inches='tight')
```

Now lets try it with `i='Fe'` and `[i]`

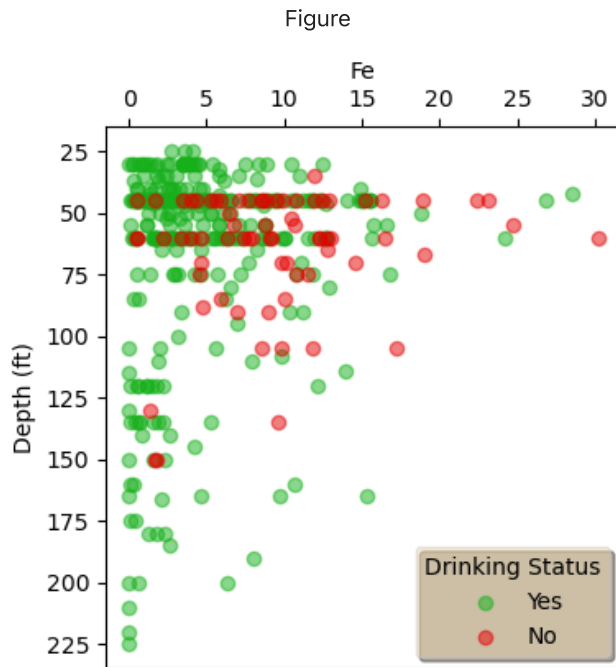
```
In [28]: fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

i='Fe'

#plot may parameters
ax.scatter(df[df.Drink=='Y'][i],df[df.Drink=='Y'].Depth
          ,alpha=0.5,label='Yes',color='xkcd:green')
ax.scatter(df[df.Drink=='N'][i],df[df.Drink=='N'].Depth
          ,alpha=0.5,label='No',color='xkcd:red')

#make look nice
ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel(i)
ax.set_ylabel('Depth (ft)')
ax.legend(loc='best',title='Drinking Status'
        ,fancybox=True,framealpha=0.5
        ,facecolor='wheat',shadow=True)

fig.savefig('Name_of_figure.jpg',dpi=200,bbox_inches='tight')
```



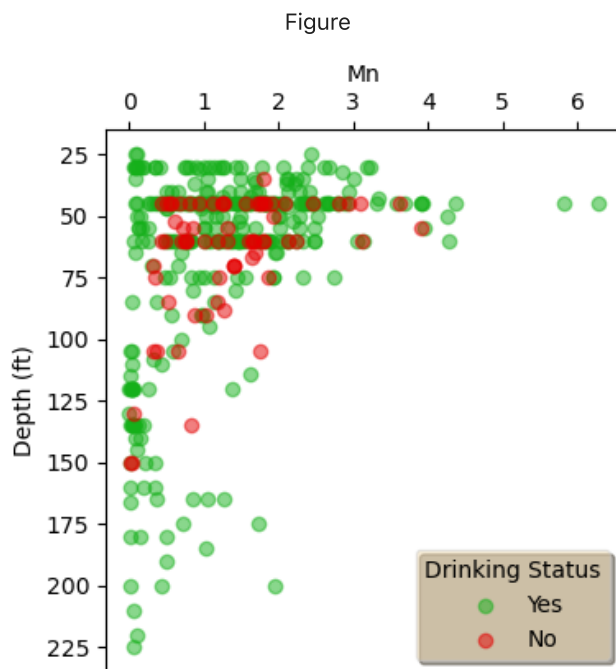
Now lets change it to a different element. Make sure your axis label is also using i so the label automatically updates

If you forget the column names...

```
In [21]: df.columns
```

```
Out[21]: Index(['Well_ID', 'Lat', 'Lon', 'Depth', 'Drink', 'Si', 'P', 'S', 'Ca', 'Fe',
               'Ba', 'Na', 'Mg', 'K', 'Mn', 'As', 'Sr', 'F', 'Cl', 'S04', 'Br'],
              dtype='object')
```

```
In [30]:
```



Goal

A plot of Depth versus every parameter.

I want to make a new file for each element. So you could name it As.jpg, Fe.jpg etc....

First just make every graph and see what happens

Combine your for loop with your plotting and make a plot of every parameter

To make it work I only did `fig,ax=plt.subplots(layout='tight')` before the for loop. Then everytime after saving clear the axes

To make the name just add the element name and .jpg

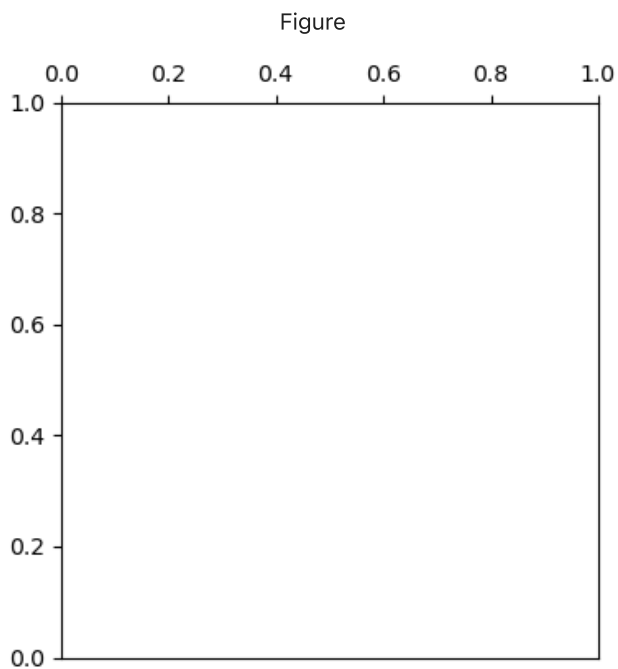
```
In [73]: fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

for column in df:

    #plot may parameters
    ax.scatter(df[df.Drink=='Y'][column],df[df.Drink=='Y'].Depth
               ,alpha=0.5,label='Yes',color='xkcd:green')
    ax.scatter(df[df.Drink=='N'][column],df[df.Drink=='N'].Depth
               ,alpha=0.5,label='No',color='xkcd:red')

    #make look nice
    ax.invert_yaxis()
    ax.xaxis.set_label_position('top')
    ax.xaxis.tick_top()
    ax.set_xlabel(column)
    ax.set_ylabel('Depth (ft)')
    ax.legend(loc='best',title='Drinking Status'
              ,fancybox=True, framealpha=0.5
              ,facecolor='wheat',shadow=True)

    fig_name='{ }.jpg'.format(column)
    fig.savefig(fig_name,dpi=200,bbox_inches='tight')
    ax.cla() #clear the axes everytime through the for loop so you can plot again
```



That was amazing! Did you look in your folder and now see all those files you made?

Look at them?

Do you know why nothing shows up on your screen?

We can do better!

I want all those figures in one pdf file instead

Read these notes carefully and take your time. The devil is in the details. READ THE NOTES!

But I want a file with all the parameters.

So we still need to learn two things.

- We know how to loop. Now lets write to a file

There are two ways to write to files and we are going to do both. They are both useful.

1. Make a one page jpg or png like we did before.
2. Make a multipage pdf.

I showed you how easy it is to make a jpg. Now lets make our multipage pdf

For a file we want pdf becuase we can do multiple pages. so if we google matplotlib write to file we get this faq page http://matplotlib.org/faq/howto_faq.html#save-multiple-plots-to-one-pdf-file But I found that not helpful. So I found this stackoverflow as the easiest. <https://stackoverflow.com/questions/21364405/saving-plots-to-pdf-files-using-matplotlib> so we need to import from matplotlib.backends.backend_pdf import PdfPages which we did above. then it is three steps.

1. import from matplotlib.backends.backend_pdf import PdfPages. we did this above
2. Open the file. pp = PdfPages('multipage.pdf')
3. save the graph. pp.savefig()
4. Close the file so we can open it somewhere else. pp.close()

so basically pp references our file and we are doing things to it. We keep adding pages to the file pp. When are done we close it.

so lets try it.

You have to close it!!!! Otherwise it gets stuck open on your computer and you can't do anything with it. If it gets stuck open just make a new file name next time you run the cell

```
In [74]: pp = PdfPages('filetest.pdf') #make and open a pdf file

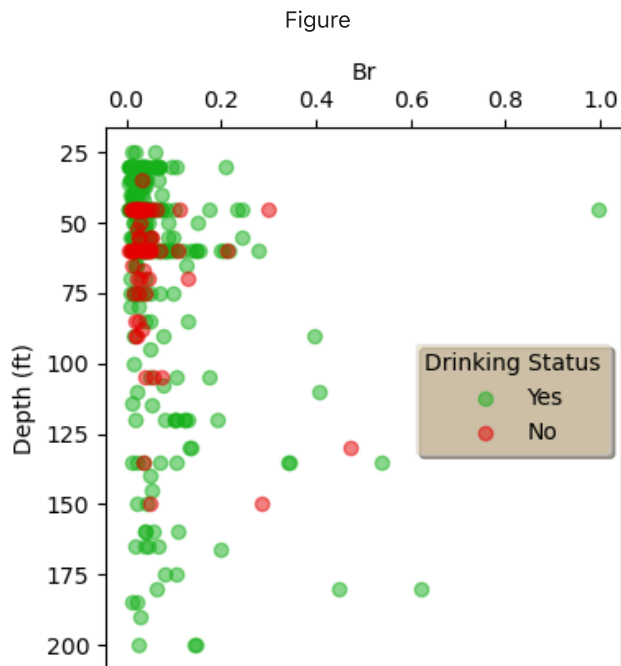
fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

#plot may parameters
ax.scatter(df[df.Drink=='Y'][column],df[df.Drink=='Y'].Depth
          ,alpha=0.5,label='Yes',color='xkcd:green')
ax.scatter(df[df.Drink=='N'][column],df[df.Drink=='N'].Depth
          ,alpha=0.5,label='No',color='xkcd:red')

#make look nice
ax.invert_yaxis()
ax.xaxis.set_label_position('top')
ax.xaxis.tick_top()
ax.set_xlabel(column)
ax.set_ylabel('Depth (ft)')
```

```
ax.legend(loc='best',title='Drinking Status'
        , fancybox=True, framealpha=0.5
        ,facecolor='wheat',shadow=True)

pp.savefig(dpi=200,bbox_inches='tight') # save to a file
pp.close() #close the file when all done
```



Important note

Hopefully you now have a file. files can get buggy. I usually have to delete the file before running the program and sometimes have to change file names. But it is nice.

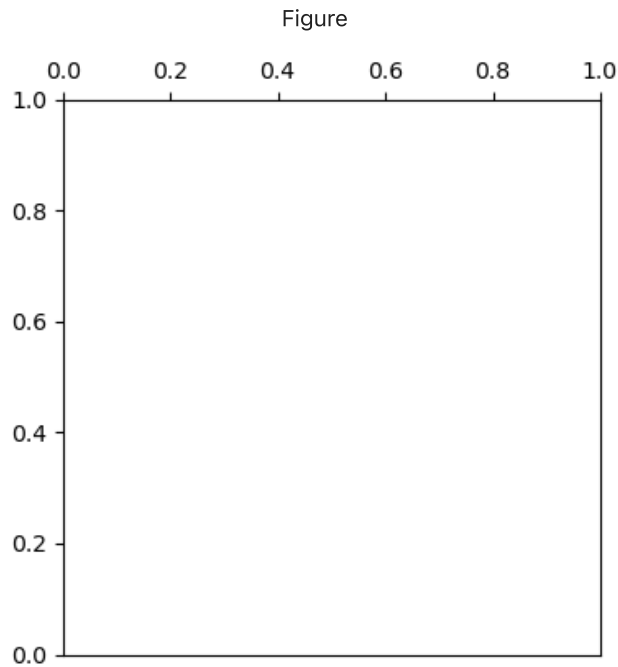
NOW PUT IT TOGETHER!!!!!!

Now you should be able to put the two parts together.

Make a pdf of every parameter versus depth.

Wait until you open the pdf file!

In [36]:



You might get an error or a crazy plot. If yes then you need to debug!

In []:

you should now have a file with all your data. I love that.

Now after looking at all the data I realize I want to plot As, Fe, S on 3 plots next to each other. Lets see if we can do this. here is an example. http://matplotlib.org/examples/pylab_examples/subplots_demo.html So if you look I want to share a y-axis. Lets try..... Lets see if we can learn. I am nervous as these plot always give me trouble

- so I going to start simple and just plot versus depth
- In subplots we will get three sets of axes. One for each plot.
- Also, I am telling it to give me 1 row by 3 columns. then we are going to share the y-axes.
- Also since we have 3 plots on one page we have 1 fig and 3 ax values.

For this work I was using i to call the element name. You can use any variable name to name the integer. Go back into your code and change i to something more intuitive. It could be j? it could be k? It could be element_name. It could be I_CANT_THINK_OF_ANYTHING It is your choice. But change the name and make it work again!

In []:

```
In [75]: fig, (ax1, ax2, ax3) = plt.subplots(1, 3, sharey=True)

column='As'
ax1.scatter(df[column],df.Depth,alpha=0.5
            ,color='xkcd:aubergine')

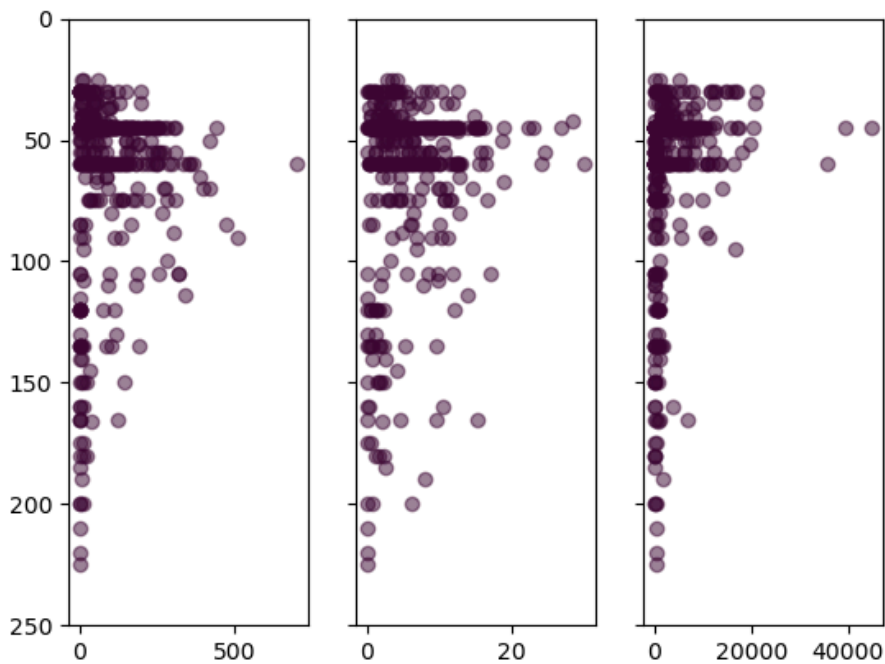
column='Fe'
ax2.scatter(df[column],df.Depth,alpha=0.5
            ,color='xkcd:aubergine')

column='S'
ax3.scatter(df[column],df.Depth,alpha=0.5
            ,color='xkcd:aubergine')

ax1.set_ylim([250,0])
```

Out[75]: (250.0, 0.0)

Figure



First step is I am going to make my ax into a list/array.

Imagine we wanted 2, 5, or 10 plots. We don't want to write ax10 etc. We can make an array so we can loop over it.

```
In [76]: fig, ax = plt.subplots(1, 3, sharey=True)

column='As'
ax[0].scatter(df[column],df.Depth,alpha=0.5
              ,color='xkcd:aubergine')

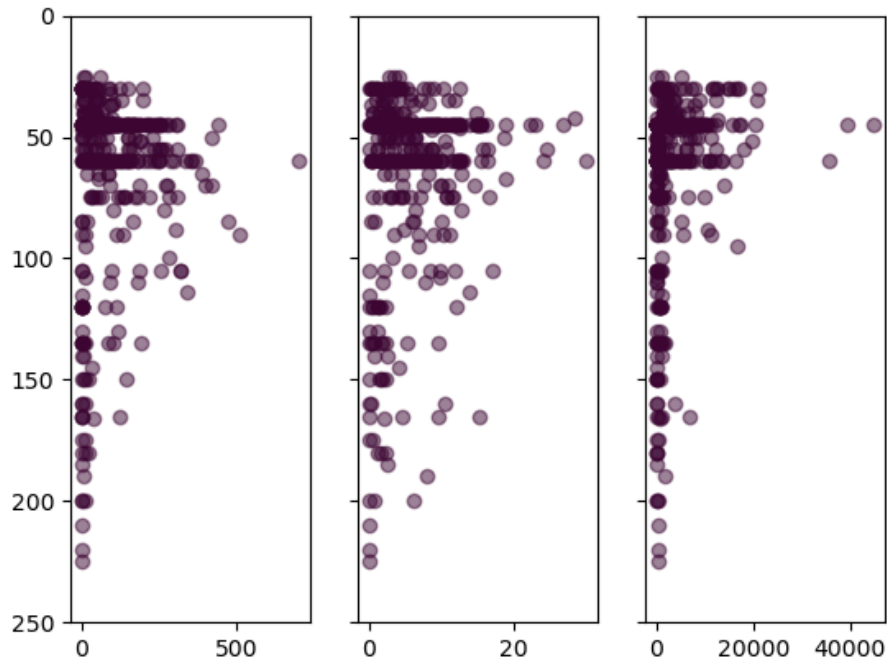
column='Fe'
ax[1].scatter(df[column],df.Depth,alpha=0.5
              ,color='xkcd:aubergine')

column='S'
ax[2].scatter(df[column],df.Depth,alpha=0.5
              ,color='xkcd:aubergine')\

ax[0].set_ylim([250,0]) #since we have sharey we only need to set one.
```

Out[76]: (250.0, 0.0)

Figure



That is really encouraging. But we have to get rid of the hardwires and do it in a for loop eventually. This way we could just choose our parameters and it would do it. So lets go back and do for loops.

```
In [49]: elements=['As','Fe','S']  
for element in elements:  
    print(element)
```

As
Fe
S

Now we could look at each of those data.

```
In [50]: elements=['As','Fe','S']  
for element in elements:  
    print(df[element])
```



```

0      NaN
1      NaN
2      NaN
3    78.97747
4      NaN
...
754    0.13000
755   17.39000
756  112.37000
757  248.93000
758      NaN
Name: As, Length: 759, dtype: float64
0      NaN
1      NaN
2      NaN
3    1.260031
4      NaN
...
754    0.341200
755    1.498350
756   22.417560
757   12.793100
758      NaN
Name: Fe, Length: 759, dtype: float64
0      NaN
1      NaN
2      NaN
3   2085.570979
4      NaN
...
754   3669.430000
755  13771.370000
756    38.300000
757   4168.520000
758      NaN
Name: S, Length: 759, dtype: float64

```

Flashback to for loops. This is what I wrote in the notes!

Python has a simple trick to count through a list and give you the list and the count. It may not make sense now but it will later in the semester!

This is the example I gave.

```
In [45]: mystrlist=['env','chem','bio','psych']
for i,mystr in enumerate(mystrlist):
    print (i,mystr)
```

```

0 env
1 chem
2 bio
3 psych

```

Now lets do it for our elements!

```
In [46]: elements=['As','Fe','S']
for count, element in enumerate(elements):
    print (count,element)
```

```

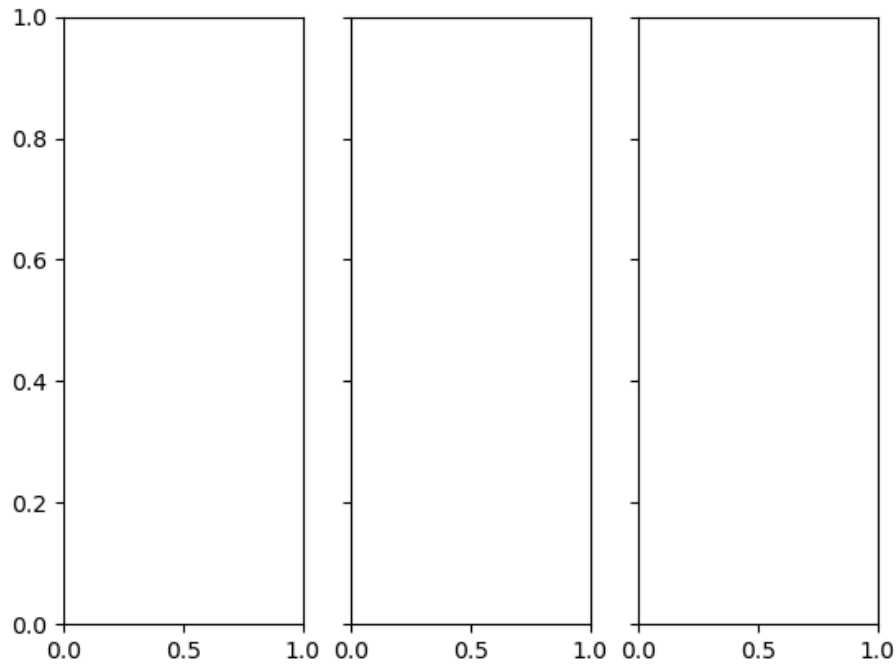
0 As
1 Fe
2 S

```

Lets keep getting rid of hard wires. Now instead of doing subplots(1,3) lets use the length of elems!

```
In [77]: elements=['As','Fe','S']  
fig, ax = plt.subplots(1, len(elements), sharey=True)
```

Figure

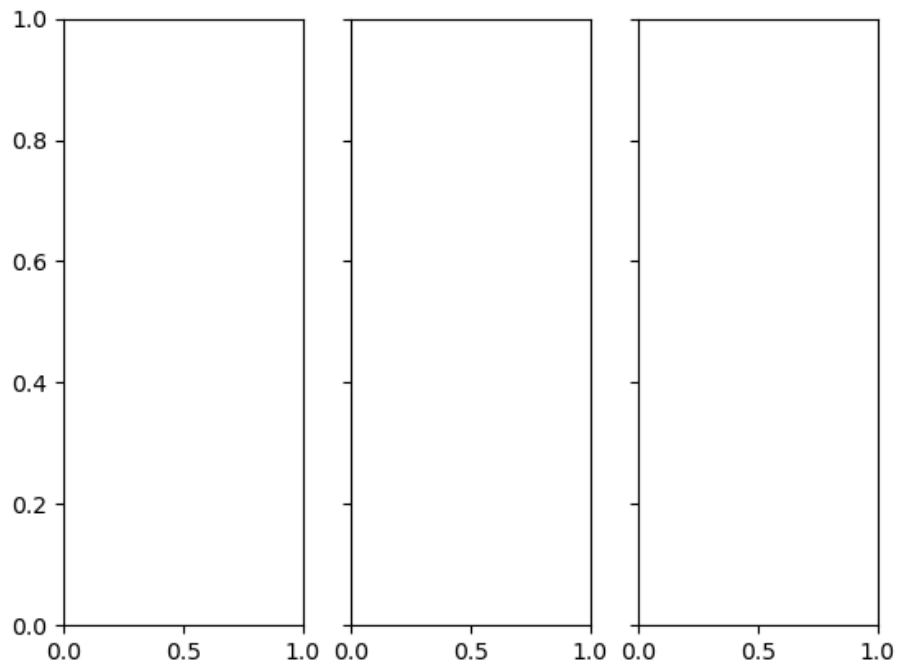


Now lets add our for loop to after we make the plots.

```
In [51]: elements=['As','Fe','S']  
fig, ax = plt.subplots(1, len(elements), sharey=True)  
for count,element in enumerate(elements):  
    print (count,element)
```

```
0 As  
1 Fe  
2 S
```

Figure

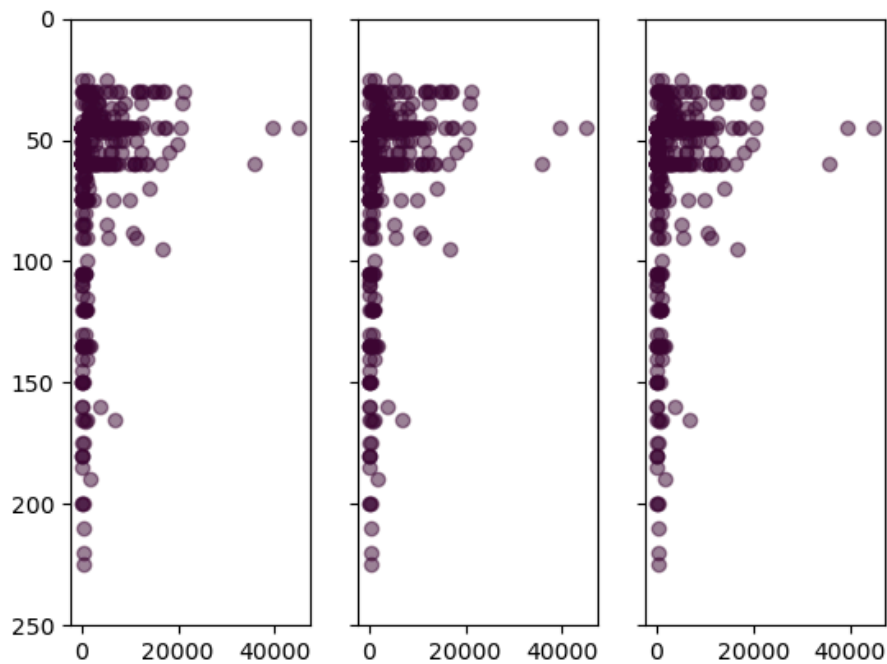


Now instead of printing lets call the correct ax and plot the elements.

In [52]:

Out[52]: (250.0, 0.0)

Figure



This is looking great. Lets see if we can clean it up. Also, lets label the y axis. and make the whole thing bigger! I use `fig.set_size_inches(width,height)` then I use `ax.locator_params(nbins=4,axis='x')` to set the number of ticks on the x axis.

```
In [59]: elements=['As','Fe','S']
fig, ax = plt.subplots(1, len(elements), sharey=True)
fig.set_size_inches(8,5)

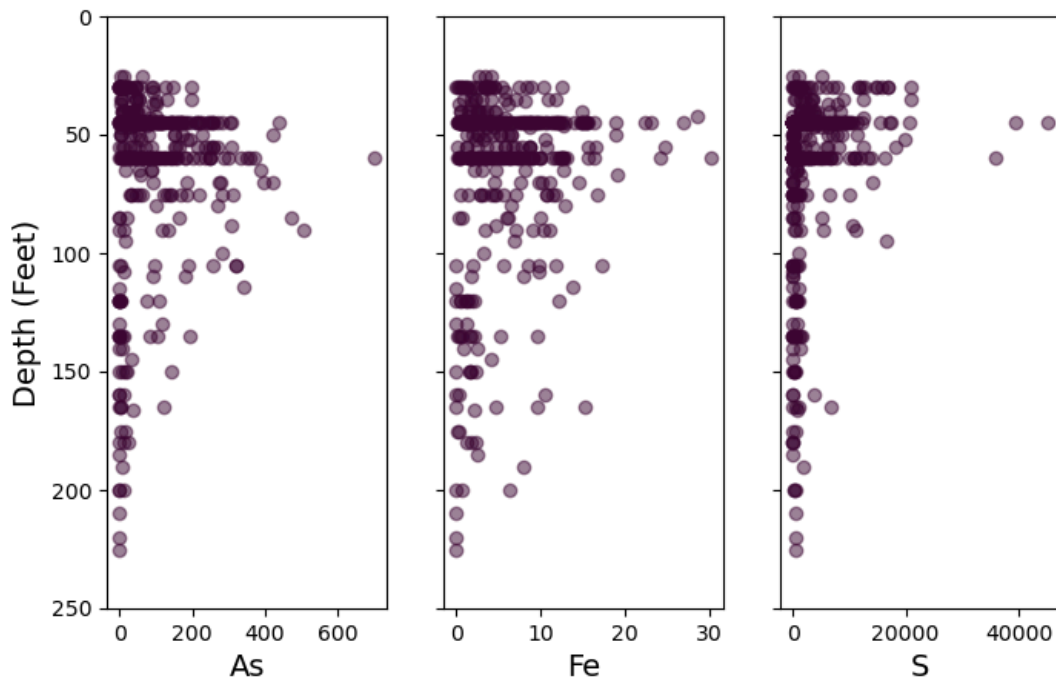
for count,element in enumerate(elements):
    ax[count].scatter(df[element],df.Depth,alpha=0.5
                    ,color='xkcd:aubergine')

    ax[count].set_xlabel(element,fontsize=14) #labels with the element
    ax[count].locator_params(nbins=4,axis='x') #gives us 4 ticks marks on the x axes

ax[0].set_ylim([250,0])
ax[0].set_ylabel('Depth (Feet)',fontsize=14)
```

Out[59]: Text(0, 0.5, 'Depth (Feet)')

Figure



That appears nicer! But beauty is in the eye of the beholder. You can choose what you think makes a nice and professional graph

If you want to publish it you need to label your graphs a,b,c, etc.

We will add these the same way we added the regression properties but instead we are going to add a letter. This is going to get weird to convert a number to a letter.

1. define your props-properties for your bbox-bounding box.
2. define your letter. this is weird. You are going to take the ascii number that is A. `ord('A')` then add our counter to it and turn it back to a letter. Adding a 0 is A, adding a 1 is a B.
3. Then we will place our letters using `ax.text`. Remember we have to transform from absolute axis to percent axis. That is what `transform` does.

4. Finally save the figure to a file. Since it is one page we will save to a jpg. The function you call is `fig.savefig('filename.jpg',dpi=600,bbox_inches = 'tight')` The dpi is dots per inch. higher is higher resolution but filesize gets bigger. I think default is 300 but higher is sometimes nice. The `bbox_inches` just makes sure nothing is cutoff. It is good to include.

```
In [62]: props=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)

elements=['As','Fe','S']
fig, ax = plt.subplots(1, len(elements), sharey=True)
fig.set_size_inches(8,5)

for count,element in enumerate(elements):
    ax[count].scatter(df[element],df.Depth,alpha=0.5
                    ,color='xkcd:aubergine')

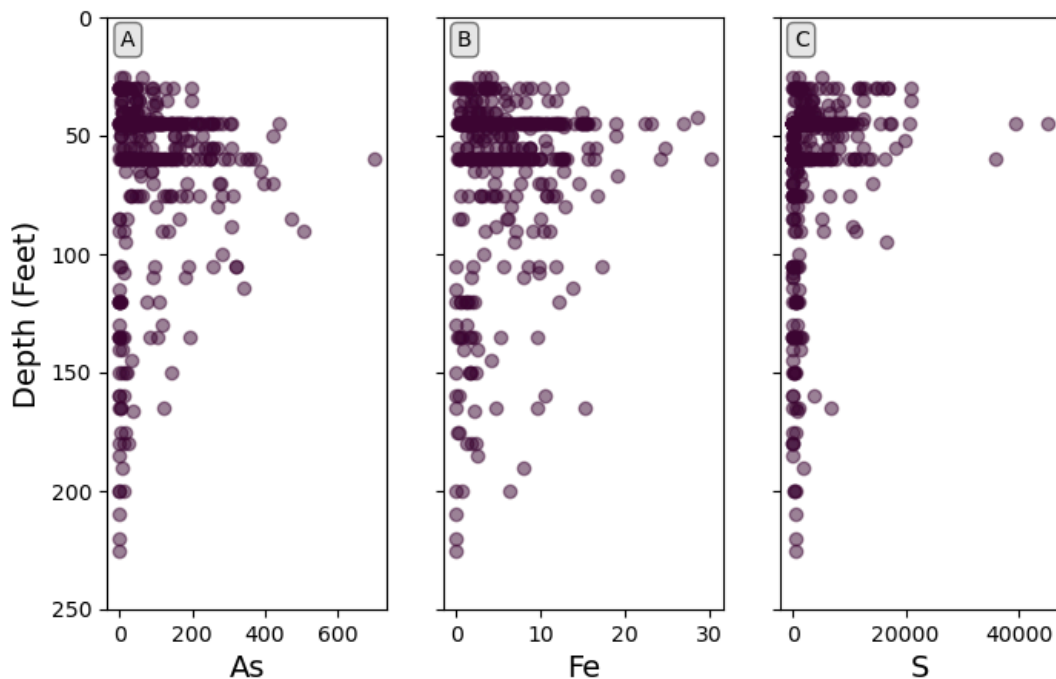
    ax[count].set_xlabel(element,fontsize=14) #labels with the element
    ax[count].locator_params(nbins=4,axis='x') #gives us 4 ticks marks on the x axes

    ax[count].text(0.05,0.98,chr(count + ord('A')),transform=ax[count].transAxes\
                  ,fontsize=10,verticalalignment='top',bbox=props)

ax[0].set_ylim([250,0])
ax[0].set_ylabel('Depth (Feet)', fontsize=14)
```

Out[62]: Text(0, 0.5, 'Depth (Feet)')

Figure

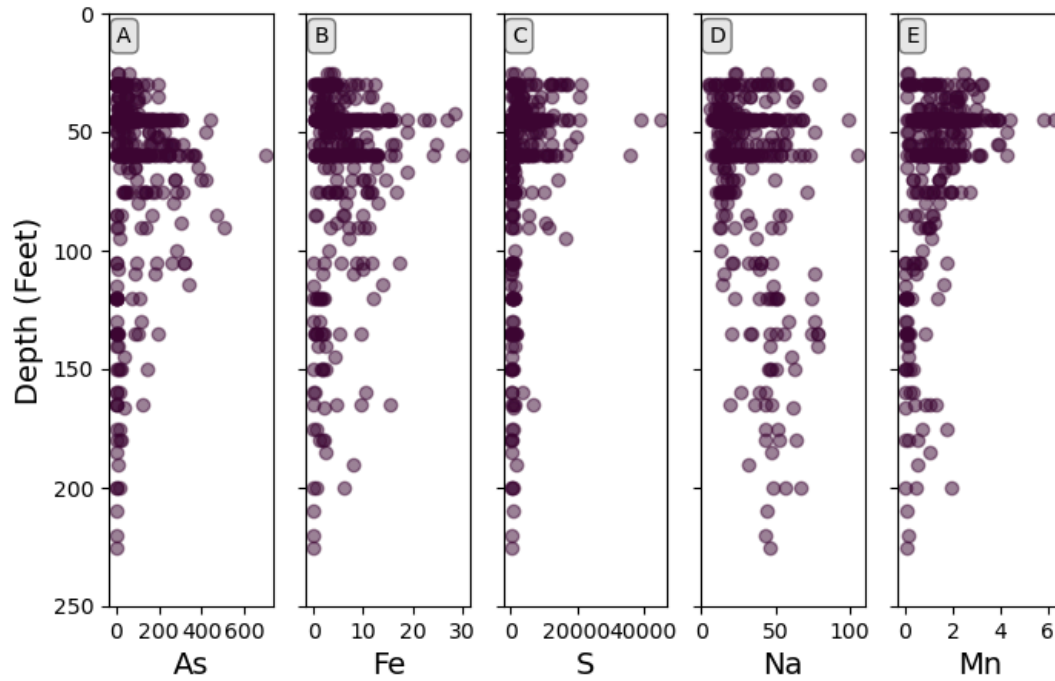


Can you change it to 5 elements?

In [63]:

Out[63]: Text(0, 0.5, 'Depth (Feet)')

Figure



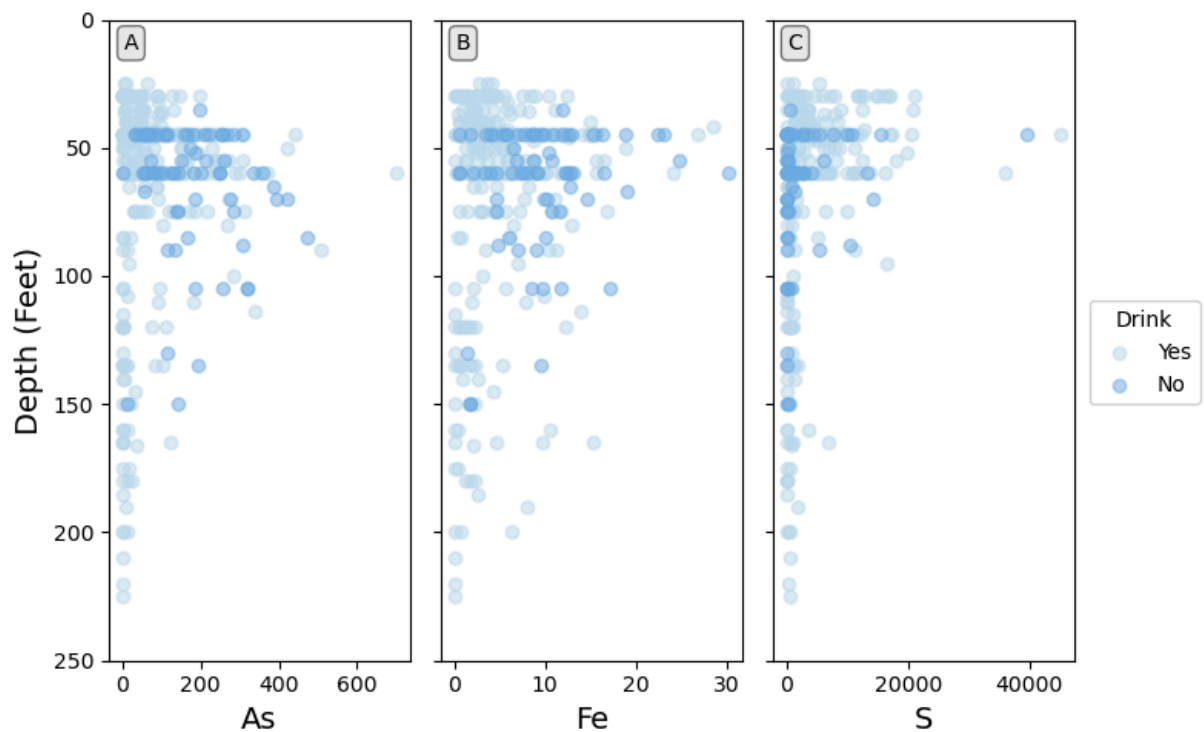
Bonus Fun.

- color the points by Drink
- add a legend
- color by columbia blues https://en.wikipedia.org/wiki/Columbia_blue

In [70]:

Out[70]: Text(0, 0.5, 'Depth (Feet)')

Figure



Homework Hints

The Drink column is going to cause scatter to crash. So you will need to avoid it. There are multiple methods. For example you could use an if statement and say

if col != 'Drink':

Then do stuff.

A more robust method might be to check and make sure the parameter is a float.

if df[col].dtype==float: then do stuff.

```
In [28]: for col in df:
         if df[col].dtype==float:
             print ('{} is a float '.format(col))
         else:
             print('{} is NOT a float'.format(col))
```

```

Well_ID is NOT a float
Lat is a float
Lon is a float
Depth is NOT a float
Drink is NOT a float
Si is a float
P is a float
S is a float
Ca is a float
Fe is a float
Ba is a float
Na is a float
Mg is a float
K is a float
Mn is a float
As is a float
Sr is a float
F is a float
Cl is a float
S04 is a float
Br is a float

```

In []:

Answers

```
In [10]: i='Drink'
         df[i].dtype
```

Out[10]: dtype('O')

```
In [12]: for col in df:
         print (col,' is of type ',df[col].dtype)
```

```

Well_ID is of type int64
Lat is of type float64
Lon is of type float64
Depth is of type int64
Drink is of type object
Si is of type float64
P is of type float64
S is of type float64
Ca is of type float64
Fe is of type float64
Ba is of type float64
Na is of type float64
Mg is of type float64
K is of type float64
Mn is of type float64
As is of type float64
Sr is of type float64
F is of type float64
Cl is of type float64
S04 is of type float64
Br is of type float64

```

```
In [14]: for col in df:
         print ('{:7s} is of type {:7s}'.format(col,str(df[col].dtype)))
```



```

Well_ID is of type int64
Lat      is of type float64
Lon      is of type float64
Depth    is of type int64
Drink    is of type object
Si       is of type float64
P        is of type float64
S        is of type float64
Ca       is of type float64
Fe       is of type float64
Ba       is of type float64
Na       is of type float64
Mg       is of type float64
K        is of type float64
Mn       is of type float64
As       is of type float64
Sr       is of type float64
F        is of type float64
Cl       is of type float64
SO4      is of type float64
Br       is of type float64

```

```

In [65]: pp = PdfPages('filetest.pdf') #open a pdf file

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

for col in df:

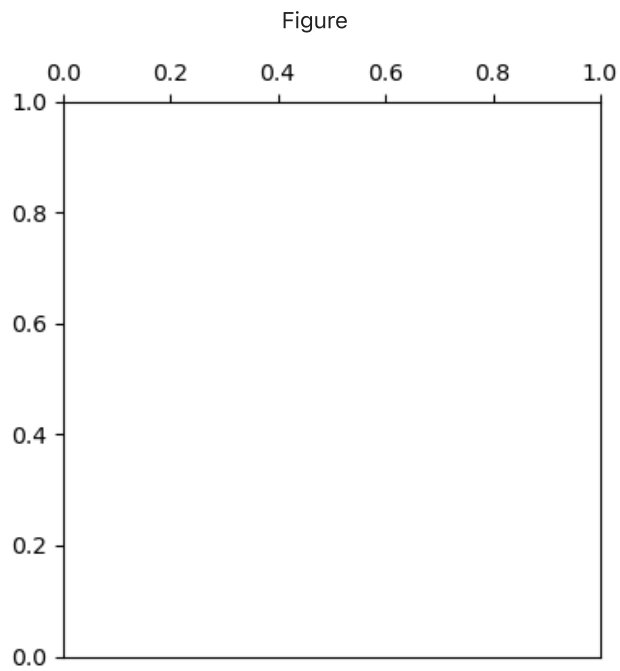
    #plot may parameters
    ax.scatter(df[df.Drink=='Y'][col],df[df.Drink=='Y'].Depth
               ,alpha=0.5,label='Yes',color='xkcd:green')
    ax.scatter(df[df.Drink=='N'][col],df[df.Drink=='N'].Depth
               ,alpha=0.5,label='No',color='xkcd:red')

    #make look nice
    ax.invert_yaxis()
    ax.xaxis.set_label_position('top')
    ax.xaxis.tick_top()
    ax.set_xlabel(col)
    ax.set_ylabel('Depth (ft)')
    ax.legend(loc='best',title='Drinking Status'
              ,fancybox=True,framealpha=0.5
              ,facecolor='wheat',shadow=True)

    pp.savefig(dpi=200,bbox_inches='tight')
    ax.cla() #clear the axes to start again

pp.close() #close the pdf file when done

```



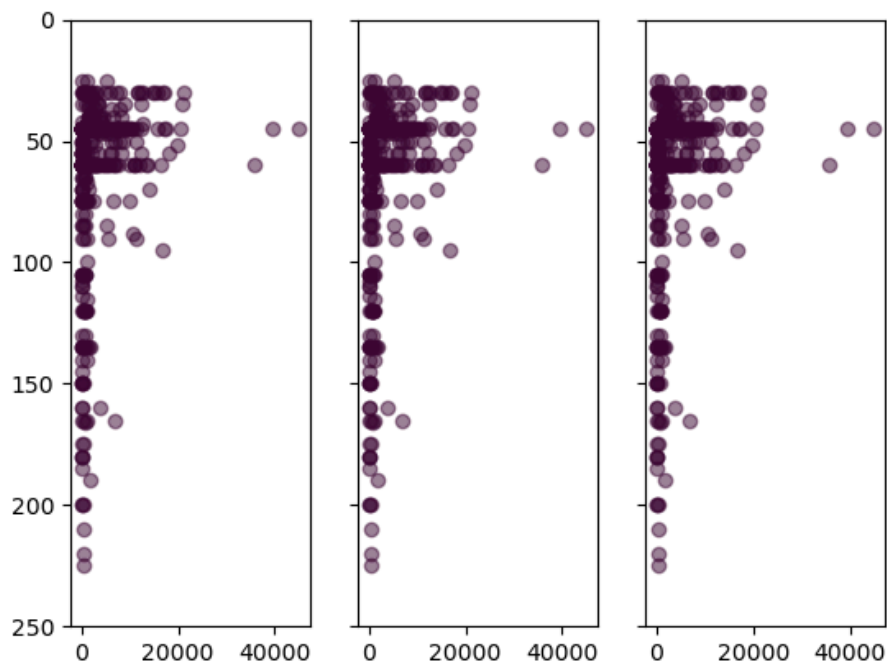
```
In [78]: elements=['As', 'Fe', 'S']
fig, ax = plt.subplots(1, len(elements), sharey=True)

for count, el in enumerate(elements):
    ax[count].scatter(df[element], df.Depth, alpha=0.5,
                    ,color='xkcd:aubergine')

ax[0].set_ylim([250,0])
```

Out[78]: (250.0, 0.0)

Figure



```
In [64]: props=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)

elements=['As','Fe','S','Na','Mn']
fig, ax = plt.subplots(1, len(elements), sharey=True)
fig.set_size_inches(8,5)

for count,element in enumerate(elements):
    ax[count].scatter(df[element],df.Depth,alpha=0.5
                    ,color='xkcd:aubergine')

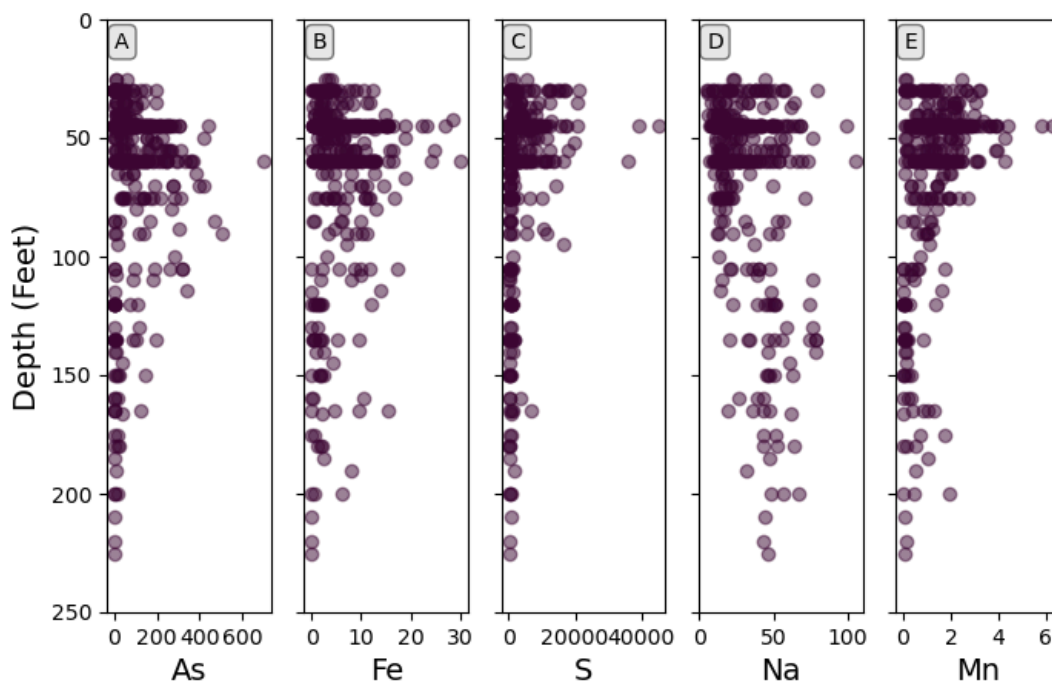
    ax[count].set_xlabel(element,fontsize=14) #labels with the element
    ax[count].locator_params(nbins=4,axis='x') #gives us 4 ticks marks on the x axes

    ax[count].text(0.05,0.98,chr(count + ord('A')),transform=ax[count].transAxes\
                  ,fontsize=10,verticalalignment='top',bbox=props)

ax[0].set_ylim([250,0])
ax[0].set_ylabel('Depth (Feet)',fontsize=14)
```

Out[64]: Text(0, 0.5, 'Depth (Feet)')

Figure



```
In [71]: props=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)

elements=['As','Fe','S']
fig, ax = plt.subplots(1, len(elements), sharey=True,layout='tight')
fig.set_size_inches(8,5)

for count,element in enumerate(elements):
    ax[count].scatter(df[df.Drink=='Y'][element],df[df.Drink=='Y'].Depth
                    ,alpha=0.5,label='Yes',color='#B9D9EB')
    ax[count].scatter(df[df.Drink=='N'][element],df[df.Drink=='N'].Depth
                    ,alpha=0.5,label='No',color='#6CACE4')

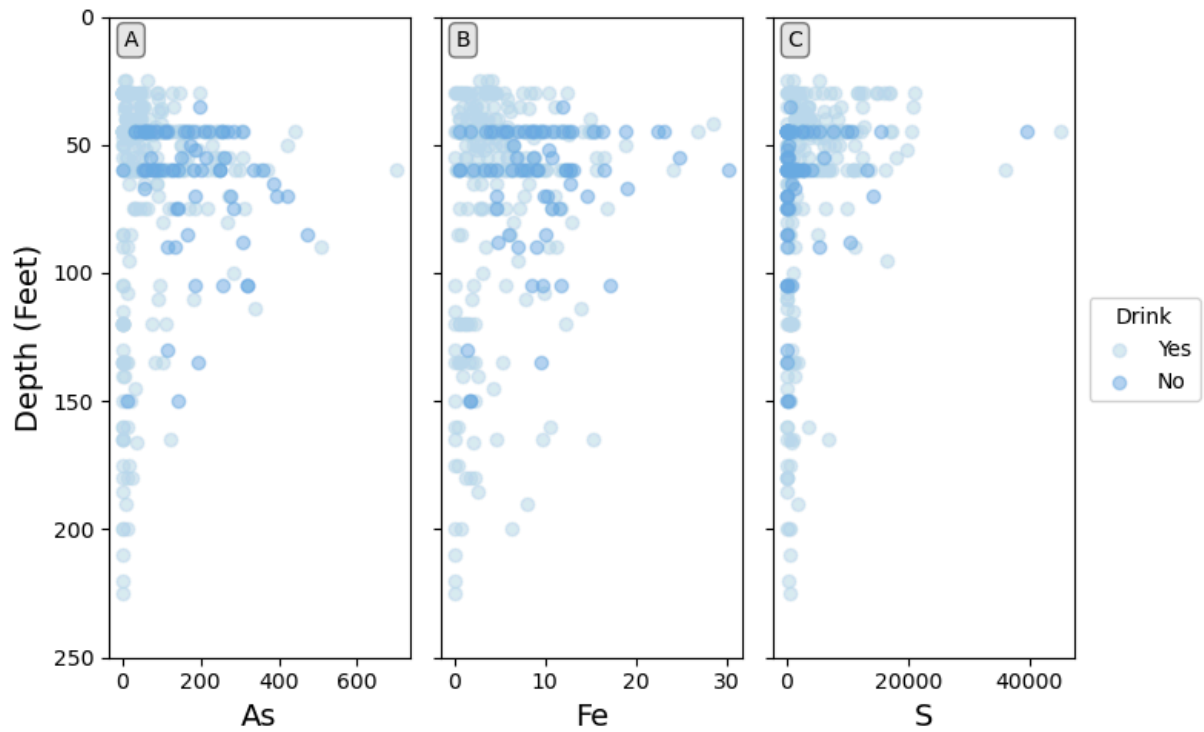
    ax[count].set_xlabel(element,fontsize=14) #labels with the element
    ax[count].locator_params(nbins=4,axis='x') #gives us 4 ticks marks on the x axes

    ax[count].text(0.05,0.98,chr(count + ord('A')),transform=ax[count].transAxes\
```

```
,fontsize=10,verticalalignment='top',bbox=props)  
ax[count].legend(title='Drink',loc=(1.05,0.4))  
ax[0].set_ylim([250,0])  
ax[0].set_ylabel('Depth (Feet)', fontsize=14)
```

Out[71]: Text(0, 0.5, 'Depth (Feet)')

Figure



In []:

In []: